

REPORT

PHASE 1:

Screenshot 1A:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

PHASE 2:

Screenshot 2A:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ find .pes/objects -type f
.pes/objects/c7/187e8fdb691b3a692e5f3f0bbcb6359e5046285225f18f9773d4fe54268c55
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/11/f3c6c0acb0a5de670c143945c8ac512af610e29774244786bf999d6798b184
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ xxd .pes/objects/c7/187e8fdb691b3a692e5f3f0bbcb6359e5046285225f18f9773d4fe54268c55 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2068   tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 002c f8d8 3d9e e295   ello.txt,...=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337   C.J.rt!.....7
00000030: 6390 25de 576d b9eb b4                      c.%.Wm...
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a          blob 6.hello.
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ xxd .pes/objects/11/f3c6c0acb0a5de670c143945c8ac512af610e29774244786bf999d6798b184 | head -20
00000000: 636f 6d6d 6974 2031 3831 0074 7265 6520   commit 181.tree
00000010: 6337 3138 3765 3866 6462 3639 3162 3361   c7187e8fdb691b3a
00000020: 3639 3265 3566 3366 3062 6263 6236 3335   692e5f3f0bbcb635
00000030: 3965 3530 3436 3238 3532 3235 6631 3866   9e5046285225f18f
00000040: 3937 3733 6434 6665 3534 3236 3863 3535   9773d4fe54268c55
00000050: 0a61 7574 686f 7220 5679 7368 616c 6920   .author Vyshali
00000060: 3c50 4553 3255 4732 3443 5336 3033 3e20   <PES2UG24CS603>
00000070: 3137 3736 3432 3338 3335 0a63 6f6d 6d69   1776423835.comm
00000080: 7474 6572 2056 7973 6861 6c69 203c 5045   tter Vyshali <PE
00000090: 5332 5547 3234 4353 3630 333e 2031 3737   S2UG24CS603> 177
000000a0: 3634 3233 3833 350a 0a43 7265 6174 6520   6423835..Create
000000b0: 7472 6565 2066 6f72 2070 6861 7365 2032   tree for phase 2
```

PHASE 3:

Screenshot 3A:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ echo "hello" > file1.txt
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ echo "world" > file2.txt
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ ./pes add file1.txt file2.txt
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
untracked:  object.c
untracked:  test_tree.c
untracked:  index.h
untracked:  Makefile
untracked:  index.c
untracked:  tree.h
untracked:  commit.c
untracked:  .gitignore
untracked:  README.md
untracked:  pes.c
untracked:  commit.h
untracked:  test_objects.c
untracked:  test_sequence.sh
untracked:  tree.c
untracked:  pes.h

vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$
```

Screenshot 3B:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776422347 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776422350 6 file2.txt
```

PHASE 4:

Screenshot 4A:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ ./pes log
commit 7a356b12ef225cb851e7b83175aa440cb47d03585c8d385d548bc78e30e51de3
Author: Vyshali <PES2UG24CS603>
Date: 1776423255

    Add farewell

commit 04daf5b07f895976deef21bc8600936bf7fe7173f375a15b0bfd810389705a75
Author: Vyshali <PES2UG24CS603>
Date: 1776423230

    Add world

commit 00f3b7b464e061b995c75d5428c56482a18fa173bc12773b6c6ff0262c41b96f
Author: Vyshali <PES2UG24CS603>
Date: 1776423173

    Initial commit
```

Screenshot 4B:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/00/f3b7b464e061b995c75d5428c56482a18fa173bc12773b6c6ff0262c41b96f
.pes/objects/04/daf5b07f895976deef21bc8600936bf7fe7173f375a15b0bfd810389705a75
.pes/objects/09/b576c2eb51b78fcf9b8493a0909522a75f8ee86df0e926312deeb5fbe7dfb1
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/5f/7a287d939df0235753fd1693fe2cd36432e4a309e39c7fb830f362b47c7aa2
.pes/objects/7a/356b12ef225cb851e7b83175aa440cb47d03585c8d385d548bc78e30e51de3
.pes/objects/84/b3e534a6e50dad999ebcaed9afd1c9a7f101d61c62c1395bbe816ceadacbb5
.pes/objects/b4/e76aee38854839557e5a3132e346adbf8288ec2ba878f2cd915d5c2326af00
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C:

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ cat .pes/refs/heads/main
7a356b12ef225cb851e7b83175aa440cb47d03585c8d385d548bc78e30e51de3
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Screenshot (Full Integration Test):

```
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed: 0eeb3ad46ce8... Initial commit

Log after first commit:
commit 0eeb3ad46ce86132a19269db186aba02ab84acd17342795fc6e773a6e2d7a368
Author: Vyshali <PES2UG24CS603>
Date:   1776424016

    Initial commit

--- Second Commit ---
Committed: cd1b6844f500... Update file.txt

--- Third Commit ---
Committed: ec628bde913... Add farewell

--- Full History ---
commit ec628bde91396522670d5bc58008b9d9861a26363f22344a598ca2b1cec5866
--- Full History ---
commit ec628bde91396522670d5bc58008b9d9861a26363f22344a598ca2b1cec5866
Author: Vyshali <PES2UG24CS603>
Date:   1776424016

    Add farewell

commit cd1b6844f50074b7f8fdb2652df7929b934a1c3f5cdc14a86ed82f496a2b6256
Author: Vyshali <PES2UG24CS603>
Date:   1776424016

    Update file.txt

commit 0eeb3ad46ce86132a19269db186aba02ab84acd17342795fc6e773a6e2d7a368
Author: Vyshali <PES2UG24CS603>
Date:   1776424016

    Initial commit

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
ec628bde91396522670d5bc58008b9d9861a26363f22344a598ca2b1cec5866

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/0e/eb3ad46ce86132a19269db186aba02ab84acd17342795fc6e773a6e2d7a368
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/cd/1b6844f50074b7f8fdb2652df7929b934a1c3f5cdc14a86ed82f496a2b6256
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/ec/628bde91396522670d5bc58008b9d9861a26363f22344a598ca2b1cec5866

=== All integration tests completed ===
vyshali@vyshali-VirtualBox:~/PES2UG24CS603-pes-vcs$
```

PHASE 5:

Q5.1:

A branch in Git (and PES) is a file inside `.pes/refs/heads/` that stores a commit hash. To implement `pes checkout <branch>`, the following changes are required:

Files to update in `.pes/`:

1. HEAD file
 - Update `.pes/HEAD` to point to:

`ref: refs/heads/<branch>`

2. No change to object store
 - Objects remain unchanged since they are immutable.

Next, the working directory must be updated to reflect the state of the selected branch. This involves reading the commit hash from the branch file, loading the corresponding tree object, and reconstructing the directory structure. All files and folders must be created according to the tree, and file contents must be restored using blob objects from the object store.

This operation is complex because it requires recursively rebuilding directories, correctly mapping object data back to files, and ensuring that existing files are safely replaced. Additionally, it must handle cases where local changes exist, as blindly overwriting files could lead to data loss.

Q5.2:

When switching branches, it is important to ensure that the user does not lose any uncommitted changes. A “dirty working directory” occurs when a file has been modified but not committed.

To detect this, we can compare the working directory with the index and the object store. The index represents the last staged version of files, while the working directory contains the current files.

For each tracked file in the index, we check its metadata such as modification time and size. If these differ from the stored values in the index, the file is considered modified. For more accuracy, we can recompute the file’s hash and compare it with the hash stored in the index.

Then, we compare the same file with the version in the target branch’s tree. If the file is both modified locally and different in the target branch, then a conflict exists.

In such a case, checkout must be refused to prevent overwriting the user’s uncommitted changes.

Q5.3:

A detached HEAD state occurs when the HEAD file contains a commit hash directly instead of pointing to a branch reference. This means the user is not currently on any branch.

If a commit is made in this state, a new commit is created, but no branch points to it. As a result, the commit is not part of any branch history and is considered “dangling” or unreachable.

Although the commit exists in the object store, it may eventually be deleted during garbage collection since no reference points to it.

To recover such commits, the user can use the commit hash (for example, from the log) and create a new branch pointing to it. This effectively reattaches the commit to the repository’s history.

PHASE 6:

Q6.1:

Over time, the object store accumulates objects that are no longer reachable from any branch. These include old commits, trees, and blobs that are not referenced by any current history.

To remove these, we perform a reachability analysis. First, we start from all branch heads stored in `.pes/refs/heads/`. Each branch points to a commit, and from that commit we can traverse its parent commits, associated tree objects, and blob objects.

While traversing, we maintain a set of reachable object hashes using a hash set for efficient lookup. This allows us to quickly mark which objects are still in use.

After marking all reachable objects, we scan the entire `.pes/objects` directory. Any object whose hash is not present in the reachable set can be safely deleted.

In a repository with around 100,000 commits and 50 branches, many commits will overlap between branches. Therefore, we would typically visit around 100,000 commits, along with their associated trees and blobs, resulting in roughly a few hundred thousand objects being examined.

Q6.2:

Running garbage collection concurrently with a commit operation is dangerous because it can lead to data loss.

Consider a scenario where a commit is in progress. The system first writes blob and tree objects to the object store, but has not yet updated the HEAD or branch reference. At this point, the new objects are not reachable from any branch.

If garbage collection runs simultaneously, it will scan for reachable objects starting from branch heads. Since the new objects are not yet referenced, GC may incorrectly identify them as unreachable and delete them.

After that, when the commit process finishes and updates HEAD, it will point to objects that no longer exist, leading to repository corruption.

Git avoids this problem by ensuring atomic operations and using locking mechanisms. Objects are written safely using temporary files and rename operations, and garbage collection is coordinated to avoid interfering with ongoing writes. Additionally, recently created objects are protected for a period of time before being eligible for deletion.